

WLMS DevOps CA-II: Step 5 Reflection & Documentation Report

Project: Warehouse Logistics Management System (WLMS) **Student:** Aman Vats, Symbiosis Institute of Technology, Pune **Course component:** DevOps CA-II, Step 5: Reflection & Report

1. Introduction

The Warehouse Logistics Management System (WLMS) is a logistics and warehouse platform with real-time inventory tracking, shipment lifecycle management and role-based access control. For this DevOps CA-II submission the application was wrapped in a complete DevOps toolchain: a GitHub Actions CI/CD pipeline, Ansible-based server configuration, Docker and Kubernetes for containerization and orchestration, and a Prometheus, Grafana and Node Exporter monitoring stack.

This report documents the architecture, the pipeline flow, the challenges faced and the lessons learned across Steps 1 to 4.

2. System Architecture

2.1 Application stack

Layer	Technology	Role
Frontend	React 19, Vite, Tailwind CSS	Single-page UI with protected routes: Dashboard, Inventory, Shipments, Deliveries, User Management
Web server	Nginx (stable-alpine)	Serves the built React bundle and falls back to <code>index.html</code> for client-side routes
Backend	Django 5.0, Django REST Framework, Python 3.11	REST API under <code>/api/auth/</code> , <code>/api/inventory/</code> and <code>/api/shipments/</code> , with JWT authentication (SimpleJWT)

Layer	Technology	Role
Database	PostgreSQL 15 (alpine)	Persistent store for users, warehouse zones, stock items, purchase orders, GRNs and dispatch orders

2.2 Domain model and access control

Three roles are enforced through a custom `User` model: **Admin**, **Warehouse Manager** and **Delivery Agent**. The data model has three Django apps:

- **accounts**: users, roles, employment status, JWT login, user management (admin only).
- **inventory**: warehouse zones and stock items.
- **shipments**: purchase orders, goods receipt notes (GRN), dispatch orders and a delivery status log. Dispatch orders move through Pending, Dispatched, In Transit, Delivered or Failed.

2.3 DevOps architecture

Concern	Tooling	Key files
Source control and CI/CD	GitHub, GitHub Actions, Docker Hub	<code>.github/workflows/devops-pipeline.yml</code>
Configuration management	Ansible	<code>ansible/playbook.yml</code> , <code>ansible/inventory.ini</code>
Containerization	Docker multi-stage builds, Docker Compose	<code>backend/Dockerfile</code> , <code>frontend/Dockerfile</code> , <code>docker-compose.yml</code>
Orchestration	Kubernetes	<code>k8s/01-postgres.yaml</code> , <code>k8s/02-backend.yaml</code> , <code>k8s/03-frontend.yaml</code>
Monitoring	Prometheus, Grafana, Node Exporter	<code>monitoring/docker-compose-monitoring.yml</code> , <code>monitoring/prometheus.yml</code>

Runtime topology (Docker Compose): the `db` container (PostgreSQL, host port 5433) is started first, the `backend` container (Django on port 8000) waits for the database and then runs migrations and seed data, and the `frontend` container (Nginx, host port 5173 mapped to container port 80) serves the UI.

Runtime topology (Kubernetes): a `db` ClusterIP service fronts a single PostgreSQL pod backed by a 1 Gi PersistentVolumeClaim. `backend-service` (ClusterIP, port 8000) fronts two backend replicas. `frontend-service` (NodePort 30080) exposes two frontend replicas to users.

3. Pipeline Flow

3.1 CI/CD with GitHub Actions

The workflow `devops-pipeline.yml` runs on every push and automates four quality gates followed by delivery:

1. **Backend check:** install Python dependencies and run Django system checks.
2. **Frontend check:** install Node dependencies and run the Vite production build to validate that the UI compiles.
3. **Ansible validation:** syntax verification of the playbook, so broken infrastructure code is caught before it reaches a server.
4. **Image build and push:** build the backend and frontend Docker images and push them to Docker Hub.

Docker Hub credentials are stored as **GitHub Secrets**, never in the repository.

3.2 End-to-end flow

Developer commit → **GitHub Actions** (Django checks, Vite build, Ansible syntax check) → **Docker Hub** (versioned images) → **Ansible** (prepares the target server) → **Docker Compose or Kubernetes** (runs the stack) → **Prometheus and Grafana** (observe it).

3.3 Configuration management with Ansible

The playbook targets the `webservers` group defined in the inventory and runs with privilege escalation. It:

- updates the apt cache and installs baseline packages (curl, git, httpd, ufw, python3-pip, ca-certificates, gnupg, lsb-release);

- creates a `devops` group and a dedicated `deployuser` account with a bash shell and sudo group membership;
- installs Docker and the Compose plugin, starts and enables the Docker service, and adds `deployuser` to the `docker` group;
- creates the application directory `/opt/warehouse_logistics` owned by `deployuser`, and writes a `.env` file with mode `0600`.

3.4 Rolling updates and rollback

Both the backend and frontend Deployments use the `RollingUpdate` strategy with `maxSurge: 1` and `maxUnavailable: 0`, so a new pod must become ready before an old one is removed, which gives zero downtime. Readiness and liveness probes on the backend gate traffic and restart unhealthy pods.

Typical demonstration commands:

- `kubectl set image deployment/frontend-deployment frontend=<new-image>` triggers a rolling update.
- `kubectl rollout status deployment/frontend-deployment` follows its progress.
- `kubectl rollout history deployment/frontend-deployment` lists revisions.
- `kubectl rollout undo deployment/frontend-deployment` rolls back to the previous revision.

3.5 Monitoring

Prometheus scrapes its own metrics and the Node Exporter every 5 seconds. Grafana (port 3000) uses Prometheus as its data source for dashboards covering host CPU, memory and uptime. Prometheus listens on 9090 and Node Exporter on 9100.

4. Challenges Faced

Challenge	How it was handled or what it taught us
Database start-up race: the Django container started before PostgreSQL accepted connections and crashed.	Added <code>wait_for_db.py</code> , which the container runs before <code>migrate</code> and <code>seed_data</code> , so start-up order is deterministic.

Challenge	How it was handled or what it taught us
Reproducible seed data: a fresh database had no users or stock to demonstrate.	A <code>seed_data</code> management command wipes and reseeds users, zones, stock, purchase orders and dispatches on start-up.
Large frontend images: building React inside the final image would ship Node and source files.	Used a multi-stage Dockerfile: Node 20 builds the bundle, and only the static output is copied into a small Nginx image.
Secrets handling: database credentials appear as plain values in <code>docker-compose.yml</code> , the Kubernetes manifests and the Ansible <code>.env</code> task.	Registry credentials are kept in GitHub Secrets. The remaining demo credentials are a known gap, to be moved to Kubernetes Secrets and Ansible Vault.
Demonstrating rollouts: the Kubernetes manifests reference generic images (<code>nginx:alpine</code> , <code>python:3.11-slim</code>) rather than the images built by the pipeline.	These are enough to demonstrate rolling updates and rollback mechanics. Pointing them at the Docker Hub images is the next integration step.
Application-level metrics: Node Exporter provides host metrics only. Latency and error-rate panels need the Django app to expose a <code>/metrics</code> endpoint.	Listed as an improvement: add <code>django-prometheus</code> and a matching scrape job.
Prometheus configuration: <code>prometheus.yml</code> contains two jobs (<code>node_exporter</code> and <code>node-exporter</code>) that scrape the same target.	Harmless but redundant; one should be removed to avoid duplicate series.
Test coverage: each Django app has only an empty <code>tests.py</code> , so the CI "checks" currently validate configuration and imports rather than behaviour.	Added to the improvement list.

5. Lessons Learned

- **Automate the boring gates first.** Running Django checks, a frontend build and an Ansible syntax check on every push catches most breakages in minutes, long before deployment.

- **Infrastructure as code removes "works on my machine".** The Ansible playbook makes a bare Ubuntu server reproducible: the same packages, the same `deployuser`, the same directory layout every time.
- **Containers need start-up discipline.** Health checks, readiness probes and a database wait script matter as much as the Dockerfile itself.
- **Zero-downtime is a configuration choice.** `maxUnavailable: 0` plus a readiness probe is what makes a rolling update safe, and `kubectl rollout undo` makes recovery a one-line operation.
- **You can only fix what you can see.** Even basic CPU, memory and uptime dashboards change how quickly problems are noticed; application metrics are the natural next layer.
- **Security is a pipeline concern.** Keeping credentials in GitHub Secrets was easy. Extending the same discipline to Kubernetes and Ansible is the main remaining work.

6. Future Improvements

1. Move all hard-coded credentials to Kubernetes Secrets, Ansible Vault and environment files excluded from Git.
2. Point the Kubernetes Deployments at the Docker Hub images produced by the pipeline, pinned to version tags instead of `latest`.
3. Instrument Django with `django-prometheus` and add latency and error-rate panels to Grafana.
4. Write real unit and API tests and make them a required CI gate.
5. Run Django behind Gunicorn instead of `runserver`, restrict CORS origins, and shorten the JWT access-token lifetime for production.
6. Add a continuous-deployment job that applies the Kubernetes manifests automatically after a successful image push.

7. Conclusion

The WLMS project shows an application taken from source code to a monitored, containerized and orchestrated deployment using GitHub Actions, Ansible, Docker, Kubernetes, Prometheus and Grafana. Beyond the tools, the main outcome is a repeatable, observable delivery process, with clear next steps for hardening it toward production.